bachelor's thesis

Detection and Recognition of Diacritical and Punctuation Marks in Real-World Images

Jan Hadáek

3rd of January 2014

prof. Jií Matas, PhD.

Czech Technical University in Prague
Faculty of Electrical Engineering, Katedra kybernetiky

BACHELOR PROJECT ASSIGNMENT

Student: Jan Hadáek
Study programme: Open Informatics
Specialisation: Computer and Information Science
Title of Bachelor Project: Detection and Recognition of Diacritical and Punctuation Marks in Real- World Images

Guidelines:

1. Consult the existing papers on text-in-the-wild recognition, focus on the issue of recognition of diacritical and punctuation marks.
2. Propose and implement a method of detection and recognition of diacritical marks in the existing text-in-the-wild recognition system [2] [3],
3. Propose and implement a method of detection and recognition of punctuation marks in the system above.
4. Suggest a method to assess a success rate of the proposed systems for detection and recognition of both diacritical marks and punctuation, evaluate the results.
5. Evaluate the influence of diacritical marks on detection and recognition of text written in Czech.

Bibliography/Sources:

1. Richard O. Duda, David G. Stork, and Peter E. Hart. Pattern classification and scene analysis. Part 1, Pattern classification. Wiley, 2 edition, November 2000.
2. Neumann L.: Vyhledání a rozpoznání textu v obrazech reálných scén. Diplomová práce, FEL VUT v Praze, 2010.
3. Neumann L., Matas J.: Text Localization in Real-world Images using Efficiently Pruned Exhaustive Search, ICDAR 2011 (Beijing, China).

Bachelor Project Supervisor: prof. Ing. Jií Matas, Ph.D.
Valid until: the end of the winter semester of academic year 2013/2014

ZADÁNÍ BAKALÁSKÉ PRÁCE

Student: Jan H a d á e k
Studijní program: Otevená informatika (bakaláský)
Obor: Informatika a poítaové vdy
Název tématu: Detekce a rozpoznání diakritických a interpunkních znamének v reálných scénách

Pokyny pro vypracování:

1. Provete reeri publikací vnovaných problematice rozpoznávání text v reálných scénách, zamte se na problematiku diakritiky a interpunkce.
2. Navrhnte a implementujte metodu detekce a rozpoznání diakritiky pro stávající systém lokalizace a rozpoznávání textu v reálných scénách popsaného v [2] [3].
3. Navrhnte a implementujte metodu detekce a rozpoznání interpunkce pro výe zmínný systém.
4. Navrhnte metodiku mení úspnosti detekce a rozpoznávání diakritických znamének a interpunkce, zmte a zhodnote dosaené výsledky.
5. Vyhodnote vliv detekce diakritiky na detekci a rozpoznání textu v etin.

Seznam odborné literatury:

1. Richard O. Duda, David G. Stork, and Peter E. Hart. Pattern classification and scene analysis. Part 1, Pattern classification. Wiley, 2 edition, November 2000.
2. Neumann L.: Vyhledání a rozpoznání textu v obrazech reálných scén. Diplomová práce, FEL VUT v Praze, 2010.
3. Neumann L., Matas J.: Text Localization in Real-world Images using Efficiently Pruned Exhaustive Search, ICDAR 2011 (Beijing, China).

Vedoucí bakaláské práce: prof. Ing. Jií Matas, Ph.D.
Platnost zadání: do konce zimního semestru 2013/2014

prof. Ing. Vladimír Maík, DrSc.
vedoucí katedry
V Praze dne 10. 1. 2013

Acknowledgement

Foremost, I would like express my gratitude to my advisor prof. Ing. Jií Matas PhD. for the time and effort he invested in sharing his immense knowledge with me. Besides that, I would like to give my thanks to Ing. Michal Buta for his endless enthusiasm and interesting discussions. Furthermore, I would like to acknowledge the professional skills of MUDr. Zdeka Janková, who helped me regain my lost concentration and stamina.

A special thank belongs to my family for their tolerance and support.

Prohláení autora práce

Prohlauji, e jsem pedloenou práci vypracoval samostatn, a e jsem uvedl vekeré pouité informaní zdroje v souladu s Metodickým pokynem o dodrování etických princip pi píprav vysokokolských závrených prací.

V Praze dne 3.1.2014

Abstract

Tato práce se zamuje na problém detekce a rozpoznání diakritických a interpunkních znamének v obrazech reálných scén. Navrhuje nové metody pro detekci a rozpoznání tchto pomocných textových symbol a experimentáln ovuje jejich úspnost. Nov navrené metody jsou integrovány do existujícího systému pro vyhledání a rozpoznání textu v obrazech reálných scén vyvíjeného v rámci Centra pro strojové vnímání na Fakult elektrotechnické eského vysokého uení technického v Praze. Tato práce rovn pedstavuje nové datové sady, speciáln vytvoené pro úely testování úspnosti detekce a rozpoznání interpunkních a diakritických znamének, a diskutuje navrhovanou metodu vyhodnocení úspnosti.

Klíová slova rozpoznávání textu; OCR; diakritika; interpunkce

Abstract

This work focuses on detection and recognition of diacritical and punctuation marks in real- world scenes. New methods of detection and recognition of these auxiliary textual symbols are proposed and the results are empirically evaluated. The proposed methods are integrated into the existing photo- ocr system being developed at the Center for Machine Perception at Faculty of Electrical Engineering of Czech Technical University in Prague. New image datasets, specially designed for the purpose of testing of detection and recognition of diacritical and punctuation marks in real- world scenes are introduced and a method of success- rate assessment is suggested.

Keywords text recognition; OCR; diacritical marks; punctuation marks

Contents

BACHELOR PROJECT ASSIGNMENT **2**

ZADÁNÍ BAKALÁSKÉ PRÁCE **4**

Acknowledgement **6**

Prohláení autora práce **7**

1 Introduction **4**
1.1 Text-in-the-wild recognition 4
1.2 Approaches in text detection 4
1.2.1 Algorithms based on image binarization 4
1.2.2 Algorithms based on edge detection 5
1.2.3 Sliding window approach 5

2 Problem specification **6**
2.1 Auxiliary textual symbols 6
2.2 Punctuation marks 7
2.2.1 Taxonomy of punctuation marks 7
2.2.2 Location of punctuation marks 8
2.3 Diacritical marks 8
2.3.1 Placement and properties of diacritical marks 9
2.3.2 Special properties of diacritical symbols 9
2.4 Integrating the OCR results with information about diacritical marks . . . 10

3 Punctuation **11**
3.1 Punctuation marks in context of a text line 11
3.2 Location of punctuation marks 12
3.2.1 Leading and trailing punctuation search windows 12
3.2.2 Embedded punctuation search windows 13
3.3 Processing of search windows 15
3.3.1 Extending the search windows 15
3.3.2 Extraction of search window images from a source image 15
3.4 Processing the information in a search window 15
3.4.1 Filtering the connected components 16
3.4.2 Punctuation pre-filter heuristics 16
3.4.3 Punctuation pre-filter classifier 17
3.4.4 Training set of the punctuation pre-filter 17
3.4.5 Grouping the connected components 18

3.5 Punctuation classifier . . . 18
3.5.1 Computation of the shape vector . . . 18
3.5.2 Dimensionality reduction . . . 19
3.5.3 The classifier . . . 19
3.6 The flow of the algorithm . . . 19

4 Diacriticals **20**
4.1 Diacritical search windows . . . 20
4.2 Segmentation of the search windows . . . 21
4.3 Detection of diacritical pairs . . . 21
4.4 Classification of diacritical regions . . . 23
4.5 The flow of the algorithm . . . 23

5 Integrating the information about diacritical marks with letter classification **25**
5.1 The implemented solution . . . 25
5.2 A discussion of an alternative solution based on HMM . . . 26

6 Datasets and testing **28**
6.1 Characteristic of the used datasets . . . 28

7 Evaluation of a diacritical pipeline **30**
7.1 Description of individual evaluation stages . . . 30
7.1.1 Procedure identifyLetter . . . 30
7.1.2 Procedure identifyDia . . . 31
7.1.3 Procedure identifyDiaRelation . . . 31
7.1.4 Check the classification and the result of rewriting . . . 31
7.2 Statistics gathered during the evaluation process . . . 32
7.3 Statistics of a dataset . . . 32
7.4 Results . . . 32
7.4.1 Dataset . . . 32
7.4.2 Performance on connected components . . . 32
7.4.3 Per-class detailed statistics . . . 32
7.4.4 Classifier confusion matrix . . . 32

8 Evaluation of a punctuation pipeline **34**
8.1 Description of individual evaluation stages . . . 34
8.1.1 Procedure inWindows . . . 34
8.1.2 Procedure findBestCCs . . . 35
8.1.3 Procedure checkGrouping . . . 35
8.2 Statistics gathered during the evaluation process . . . 36
8.3 Statistics of a dataset . . . 36
8.4 Results . . . 36
8.4.1 Dataset . . . 36
8.4.2 Detection performance on connected components . . . 36
8.4.3 Number of true positive regions at different stages . . . 36

9 Implementation **38**
9.1 Diacritical module 38
9.2 Punctuation module 39
9.3 Training datasets of classifiers 39

10 Conclusion **40**

A Dataset format **42**

B Annotation editor **45**
B.1 Semi-automatic annotation 45

C Content of the enclosed CD **46**

D A visual preview of the datasets **47**
D.1 CSDataset2 47
D.2 CSDataset4 47

E A visual preview of detection results **48**
E.1 Diacritical pipeline 48
E.2 Punctuation pipeline 48

Chapter 1

Introduction

1.1 Text-in-the-wild recognition

The problem of text- in- the- wild recognition, also known as photo- ocr or real- scene text recognition, is a challenging problem of computer vision. There are specific issues in text- in- the wild recognition, that make it more difficult than recognition of printed documents, which is considered to be a virtually solved problem.

Common typographical conventions present in printed documents are often ignored in real- scene labels and posters, which complicates the process of text- line estimation and decision about word boundaries. Geometric distortions caused by a camera photographing labels from various angles are present, as well as distortions introduced by a camera lens itself. A real- scene photograph features various illumination conditions. It also contains less text than scanned documents and the scale of text regions is more varied, which, together with the presence of many potential false positives in real- world images, complicates the problem of text detection.

1.2 Approaches in text detection

The problem of detection of auxiliary textual symbols is is closely related to detection of textual regions. There are several approaches how text regions can be detected. Each of them impose it's own strengths and weaknesses. Here the most common approaches will be listed and their features will be discussed:

Image binarization (thresholding), Edge detection, Sliding windows.

1.2.1 Algorithms based on image binarization

Methods of auxiliary textual symbols detection and recognition proposed in this manuscript are designed to be incorporated in a system using a method based on image thresholding, in particular the TextSpotter system, which uses a detector called CSER (described more in details in [1][2] and [3]). All methods based on image binarization share the similar advantages and disadvantages.

The key advantage of a method based on image binarization is that scale invariance is easy to achieve because textual regions of all scales and rotations can be extracted from an image in a single pass. The main disadvantage is that this process produces a set of connected components (blobs) and ignores any relations between these blobs completely. If a glyph consists of more blobs, it is necessary to relate them somehow, and if any of

them can get lost during the process of binarization, it is impossible to recover it without a complementary method. The method is dependent on the fact that most of the letter regions consist of areas having uniform color (single- color ink) and may have problems with textured letters or backgrounds.

Figure 1.1: Stenciled labels (as well as segment displays, for example) impose a problem to methods based on blob extraction, but are directly detectable by sliding windows. [6]

1.2.2 Algorithms based on edge detection

Methods exploiting edge detection are based on the assumption that a relatively steep image gradient is present on borders of letters and numerals. Such gradient can be detected by a standard edge detector (e.g. Canny). The most prominent method utilizing the edge detection is the Stroke Width Transform described in [4]. The final output of this method is a set of connected components, similar to methods based on image binarization, however, the blobs are not obtained directly. A sophisticated algorithm utilizing the notion of Stroke Width consistency is used to transform the edges into component components.

This method is more robust to textured ink and background than image binarization. It performs poorly on blurred glyphs because they are not featuring significant edges to separate a glyph from its background. Also, the SWT may fail in some outlined fonts because it has difficulties to distinguish between a glyph itself and its outline.

1.2.3 Sliding window approach

Sliding windows have been utilized in several works, e.g. [5]. Sliding window detectors have an advantage of being potentially more robust in situations that involve connectivity defects than the methods mentioned above.

Letters may contain defects resulting that an originally single- component glyph can be split into multiple connected components (see figure 1.1). Such letters will not be extracted by an image binarization technique as a single object (unless some explicit post-processing technique to merge such letters is used). The technique of sliding windows can directly extract multi- component letters, which simplifies their further processing.

To achieve some geometric invariance, a detector based on sliding windows scans the image multiple times, each time using different fixed properties (dimensions, aspect ratios, rotations) of the windows. This results in comparatively slow running times and limited geometric invariance.

Chapter 2

Problem specification

2.1 Auxiliary textual symbols

This work focuses on diacritical and punctuation marks, both being what we call auxiliary textual symbols, particularly their detection and recognition in photo- ocr systems utilizing a text detector based on image binarization.

Common textual information written in Roman script consists of letters, Arabic numerals and auxiliary textual symbols, which have several specific features that differentiate them from letters and numerals:

- **Semantic differences.** Every Roman letter (or a group of them) represents a sound in a spoken form of a language; Arabic numerals are ideograms, they represent the notion of a number. Semantics of auxiliary textual symbols is more contextdependent than semantics of letters since the auxiliary textual symbols only modify the meaning of letters, words or whole sentences located on the same text line.
- **Geometry.** Both numerals and letters have comparable dimensions and a standard placement (a text- line). Dimensions and placement of auxiliary textual symbols are more varied than dimensions and placement of letters. Auxiliary textual symbols are often smaller than letters which makes their detection and recognition more challenging.
- **Topology.** Glyphs of all Roman letters and numerals consist of a single connected component (blob). Some auxiliary textual symbols may be composed of two, three or a greater number of connected components.

The goal of this work is used to extend the TextSpotter system [1], which is a real scene text localization and recognition system, being developed at Czech Technical University,

Figure 2.1: A segmented text from a real scene, having the auxiliary textual regions highlighted. Diacriticals are depicted in green, punctuation marks in magenta.

and to add support for auxiliary textual symbols. The TextSpotter is an end- to- end solution, and the flow of the algorithm it uses can be divided into three basic steps:

1. Character segmentation and detection (based on image binarization),
2. Grouping and ordering of detected regions into lines and words,

3. Recognition of detected regions.

This flow has an influence on detection and recognition of auxiliary textual symbols and chosen strategies presented in this work:

1. During the first step, thousands of connected components are detected and a fast classifier based on sequentially computable features is used to distinguish between "useful" (potentially letter-forming) and "garbage" blobs. The classifier is trained to accept letters and numbers only. Extending the training set with more symbols, especially simple ones, would likely increase the occurrence of false positives.

2. The method of grouping the detected CSER regions into lines and breaking them up into words is already quite complicated and computationally costly. Only letters and Arabic numerals are considered at this point. Trying to incorporate the support for auxiliary textual symbols, especially these that do not have letter-like dimensions or topology, would possibly make this process extremely computationally expensive with current algorithm.

Auxiliary textual symbols are related to the surrounding text. Since the TextSpotter is able to recover this text, we can exploit the information about the text to search for auxiliary textual symbols effectively. This is the reason why the post- processing strategy was chosen for auxiliary textual symbols recognition and detection.

2.2 Punctuation marks

Punctuation marks are symbols that indicate the structure and organization of written language, as well as intonation and pauses to be observed when read aloud [7]. This definition captures the typographical meaning of dashes, hyphens, commas, colons, brackets and parentheses. It may not be entirely correct from the typographer's point of view, however, for the purpose of this work, not only the objects mentioned above, but all typographical symbols that form a part of the written text will be called punctuation marks. Examples include the "at" sign, percent sign, hash sign, underscore, currency symbols and so on.

Punctuation marks can be a good source of semantic information about sentences. They express the context of words, meaning of sentences and separate clauses in compound sentences. This information is very useful for machine translation algorithms or advanced voice synthesis software.

2.2.1 Taxonomy of punctuation marks

From a technical (not semantic) point of view, it's possible to taxonomize the punctuation according to several criteria:

- **Position and dimensions (relative to the line).**
  - Letter-like dimensions and location (for example &, $, #, braces, ...).
  - Small, located near to the bottom line (dot, comma).
  - Small, located near to the mean line (dashes, hyphens).
  - Small, located near to the top line (apostrophe, single and double quotes).

- **Connectivity**
  - A single connected component (most of the letter- like symbols).
  - Two connected components (colon, semicolon, question mark, exclamation mark, equal sign, ...).
  - More connected components (sometimes the percent sign, special mathematical symbols).

Figure 2.2: Visual demonstration of punctuation taxonomy. Left to right, top to bottom: at, ampersand, dollar, slash, hash, section sign, dot, comma, hyphen, dash, tilde, apostrophe, prime, single quotes, backtick, normal and inverted exclamation mark, normal and inverted question mark, non-English quotes, colon, semicolon, guillemets, double quotes, double prime.

It is vital to mention the problem of connectivity here since many punctuation marks consist of more than one connected components. If a punctuation glyph consists of a single connected component (and has letter- like dimensions and placement), it can be localized and recognized by the algorithms developed for letters. However, the TextSpotter doesn't support multi- component letters, so it's important to address the issue of disconnectedness somehow, in order to be able to locate and recognize punctuation marks.

2.2.2 Location of punctuation marks

Figure 2.3: Real-world examples of different kinds of punctuation placement.

Punctuation marks always appear either on the beginning or end of a text line or between two words, always outside of the word borders[1]. There are three basic locations where a punctuation mark can appear:

- In front of a text line, for example a hyphen or a quote starting a sentence (which will be called **leading punctuation**).
- In between of two neighbour words (which will be called **embedded punctuation**).
- On the very end of a text line, (which will be called **trailing punctuation**).

2.3 Diacritical marks

Diacritical marks are the accent marks used on some characters to denote a specific pronunciation. Rare in English, they are a common occurrence in French, German, Italian, Spanish, Czech and other languages. Some of the more commonly seen diacriticals include acute, caron, cedilla, circumflex, grave, tilde, and umlaut.[8] Also, the dot symbol (also called a tittle) above lowercase letters "j" and "i" can be considered a diacritical mark.[9]

[1]This assumption might be invalid for some languages, but it works for all languages using Roman script.

Although dots have no semantic importance in most languages, (the exception is Turkish, for example, which uses both dotted and dot- less "I" [10]), they have similar geometric properties and placement as other diacriticals and information about their presence (or absence) could be exploited to aid the OCR to produce better results in some cases (eg. disambiguating lower- case "i" from capital "l").

Simmilarly to punctuation marks, diacriticals are objects of semantic importance. Omission or negligence of diacriticals creates ambiguities that cannot be resolved based on a language model alone and are likely to cause troubles in further processing (full-text search, machine translation, etc.). In Czech language, good examples of ambiguities are pairs like zvykat (to be getting used to) Œ výkat (to chew) or med (honey) Œ md (copper) and many other pairs of words that are spelled the same if diacriticals are omitted. Sometimes even human beings cannot disambiguate a sentence without diacriticals reliably, which can result in misunderstanding.

In Czech language, there are three kinds of diacritical marks: acute, caron and ring. Additionally, the caron symbol has two forms, depending on the letter which belongs to it. We will call this glyph, which only appears in letters d and t a hook caron", because, for the sake of detection and recognition, it would be a distinct object with distinct properties.

Figure 2.4: Example of diacriticals in Czech language.

2.3.1 Placement and properties of diacritical marks

Diacritical symbols in Czech language, as well as many other languages that use diacriticals, are placed above letters, whose meaning they modify. We assume that a text- in-the- wild recognition system is able to provide us with information about individual words and letters that have been found (i.e. the detection and recognition of diacritical marks is implemented as a post- processing stage). This information will

2.3.2 Special properties of diacritical symbols

Diacriticals clearly share some features with punctuation marks. They also modify the meaning of letters and have different meanings and dimensions compared to letters. However, there are substantial differences between diacriticals and punctuation symbols too, so a slightly different approach has been taken

One of the important differences is the fact that a diacritical mark is bound to a letter. A letter with a diacritical symbol can actually be considered to form a separate atomic alphabetical symbol. In fact, it is the most common way how computer fonts interpret letters with diacriticals. However, there are good reasons why this approach can be impractical in photo- ocr systems:

- Complementary source of information about letters. If the diacriticals are dealt (detected and classified) partially independently, they can be used to improve the OCR classification of letters. This idea will be extended in Chapter 5.

2.4 Integrating the OCR results with information about diacritical marks

Even if a reasonably reliable algorithm is able, it is not trivial to use the information obtained by the OCR output. Assume we have a set of detected and recognized letters and a set of diacritical pairs as defined in Chapter 4. Both letter recognition and diacritical detection/recognition have some degree of uncertainty and may be contradictory.

As mentioned in the previous section, there is a need to detect and recognize diacriticals as available, it is not trivial to use the information obtained by the OCR output. Assume we have a set of detected and recognized letters and a set of diacritical pairs as defined in Chapter 4. Both letter recognition and diacritical detection/recognition have some degree of uncertainty and may be contradictory.

It is clear that knowledge about a letter can improve the detection/recognition of a punctuation symbol and vice versa. In this work, a simple heuristic solution of this problem will be implemented, and a possibility of implementing a more sophisticated solution incorporating a language model will be discussed.

Chapter 3

Punctuation

This chapter describes the strategy employed in detection and recognition of punctuation marks. The algorithm works with the following data:

Input: a real- scene image a list of detected textlines, words and letters (their locations and segmentation)

Output: a list of detected punctuation regions along with their classification

The employed strategy consists of several steps:

1. Define the exact regions where the punctuation marks are expected to occur and extract the regions from the image. These regions will be called punctuation search windows.
2. Extract connected components from the windows.
3. Classify each of the extracted components and discard those that are unlikely to be a part of a punctuation mark.
4. Use a heuristic rule to group the extracted components together.
5. Classify the resulting groups. Discard those that are too ambiguous as they are likely to be false positives.
6. Add the resulting punctuation marks to TextSpotter OCR output.

3.1 Punctuation marks in context of a text line

First, it is important to introduce a model of an ideal text line, which is used by the TextSpotter system. Such text line can be described by it's baseline (possibly slanted, but otherwise a straight line segment) and few measurements, which induce other lines parallel to the baseline: capital letter height (top line), x- letter height (mean line), descent (descender height) and ascent (ascender height). Such textline is depicted on the Figure 3.1.

Figure 3.1: A text line. [11]

A text line consists of one or more words. In this thesis we define a word as a sequence of letter regions separated from other words on the same line by a white space or a

non- letter region. A letter region is any region that can be classified as a letter by the TextSpotter system, including lower- case letters, upper- case letters and Arabic numerals. In accordance with the definition of a text line model, we can assume that every word can be bounded by a rotated rectangular bounding box, whose bottom and top sides are parallel to the base line.

3.2 Location of punctuation marks

Punctuation marks are always located within the area of a text line. The punctuation recognition method is based on the assumption that a text recognition system can provide us with information about text lines that can be exploited for the purpose, eg. the slant of a text line as well as the position of a top line. Let's call the area in between the bottom line and top line a text line region and this ideal text line a geometric text line.

As mentioned in the Section 2.2.2, punctuation can be divided into three groups according to its placement with the respect of surrounding words:

- Leading punctuation,
- trailing punctuation,
- embedded punctuation.

3.2.1 Leading and trailing punctuation search windows

Figure 3.2: A visualisation of leading and trailing punctuation search windows. The search windows as defined in this section are depicted as solid red rectangles; the extended search windows (Section 3.3.1) are depicted as dotted line rectangles.

Leading [trailing] punctuation is the punctuation located at the end [beginning] of a text line. Semantically, dashes, quotes or parentheses are likely to be spotted as leading punctuation marks while dots, colons, exclamation marks etc. are likely to be located on the end of the line. For leading and trailing punctuation, we only have the information about one word available, and, therefore, the span of the window is unknown on one of the sides.

In order to capture the leading [trailing] punctuation, we need to extrapolate the information known for the first [last] word of a text line. It is reasonable to expect that the punctuation will be located within the text line area defined by the geometric text line of the respective word. The height and text line slant are readily available. The only unknown parameter is the width of this area.

We want to maximize the chance that every punctuation region that belongs to a given word will be captured. However the further we extend the search window, the greater are the chances that additional unwanted regions will be captured and these may introduce false- positives during the process. The question how far to extend the search windows is an open question. An empirical estimate of the optimal width of leading [trailing] punctuation search windows is around 1.5 of text line height, and this value has been used in the testing.

3.2.2 Embedded punctuation search windows

In this kind of punctuation, we have two words surrounding words available and we will use the information known about them to derive a punctuation search window. Before we define search windows in between two words more in detail, it is essential to introduce the notion of neighbouring words.

Semantic text lines and neighbouring words

> **Rule 3.2.1.** Two words are neighbouring each other if they follow each other immediately on the same semantic text line.

A semantic text line determines a semantic (left- to- right) ordering of letters and other objects. The approximation of a text line by a strictly straight geometric text line does not always correspond with reality. Therefore, what's perceived by a human as a single line of text, might end up being split into more lines in the internal representation of a text recognition system, which is using the notion of a text line that has been previously described. Each of the artificially introduced text lines would contain one or more words of the text line as perceived by a human. To address this issue, a heuristic post- processing stage is employed in the system, and the original lines are grouped together into longer ones.

Figure 3.3: Relative mutual vertical distance. The red arrows mark the distances considered. The maximum of the depicted distances is used as a final result.

The task is to merge more geometric text lines into one semantic text line. The heuristic rule is the following: Assume we have two geometric text lines, $\mathbf{L}_1$ and $\mathbf{L}_2$, that w_1 and w_2 are words and $\mathbf{x}(w)$ is the maximum x coordinate of a word w . Furthermore, assume that $\forall \mathbf{w}_1 \in \mathbf{L}_1 \vee \mathbf{w}_2 \in \mathbf{L}_2$ holds that $\mathbf{x}(\mathbf{w}_1) < \mathbf{x}(\mathbf{w}_2)$. In other words, the $\mathbf{L}_1$ is on the left side relative to $\mathbf{L}_2$. The two geometric text lines are grouped into one semantic line based on the notion of relative mutual vertical distance.

The relative mutual vertical distance is computed as follows: We consider the rightmost word of $\mathbf{L}_1$, let's call it $\mathbf{w}_1$ and the leftmost word of $\mathbf{L}_2$, denoted as $\mathbf{w}_2$. We take into account the bottom- left and bottom- right points of the bounding boxes of both words. Let's denote them $\mathbf{L}_1$, $\mathbf{R}_1$ (for the word $\mathbf{w}_1$) and $\mathbf{L}_2$, $\mathbf{R}_2$ (for the word $\mathbf{w}_2$). For each of $\{\mathbf{L}_1, \mathbf{R}_1\}$, we compute a vertical (y- axis) distance from the line $\mathbf{L}_1$. Likewise, for each of $\{\mathbf{L}_2, \mathbf{R}_2\}$, we compute a vertical (y- axis) distance from the line $\mathbf{L}_2$. The procedure is illustrated on the Figure 3.3. The resulting relative mutual vertical distance is computed as maximum of these distances divided by minimum of heights of $\mathbf{L}_1$ and $\mathbf{L}_2$.

> **Rule 3.2.2.** Two geometric lines are grouped together into one semantic line if they satisfy the constrains above and if the relative mutual vertical distance is less than 0.5.

The choice of this value is arbitrary and based on experience and expectations what humans consider to be a single text line. It may be questionable, but in practice this definition of a semantic text line seems to be suffice.

Definition of embedded punctuation windows

Assuming two neighbouring words $\mathbf{w}_1$ and $\mathbf{w}_2$, $\mathbf{w}_1$ located left of $\mathbf{w}_2$ located on the same semantic text line, the goal is to define a search window in between of these words, where the punctuation marks shall be sought. The search window is defined by a rotated rectangle:

Figure 3.4: Embedded punctuation search window. The slant difference is extremely exaggerated so that the principle is more visible. The search window as defined in this section is depicted as a red solid rectangle; the extended search window (Section 3.3.1) is depicted as a dotted line rectangle.

1. The bottom text line of both word regions is known, as well as the height of the letters (which is effectively defining the top text line as well).

2. Construct a tight, rotated rectangular bounding box for each of the words. The bottom side of the rotated rectangle lies on the bottom text line of the given word, the top side of the rectangle lies on the top text line. The left [right] side is orthogonal to the bottom text line and touching (but not intersecting) the mask of the leftmost [rightmost] letter of the word on the left [right] side.

3. Denote the lower-right point in the rotated bounding box of $\mathbf{w}_1$ by B_1 and the upper-right point in the same bounding box by T_1 . Likewise, denote the lower-left point of the rotated bounding box belonging to $\mathbf{w}_2$ by B_2 and the respective upper-left point by T_2 .

4. Denote a line passing through the points B_1 and B_2 by b . This line defines the bottom line of the search window.

5. Compute the point-to-line distances $d(\mathrm{T}_1, b)$ and $d(\mathrm{T}_2, b)$. If $d(\mathrm{T}_1, b) > d(\mathrm{T}_2, b)$, denote the point T_1 by F , otherwise denote T_2 by F .

6. Construct a line f , which is passing through the point F and is parallel to b . This line defines the upper boundary of the search window.

7. Find points F_1 and F_2 . The point F_1 [F2] is defined as an intersection of the top line f and the line defined by points $\mathrm{B}_1\mathrm{T}_1$ [resp. $\mathrm{B}_2\mathrm{T}_2$]. The search window is now defined by a rotated trapezoid $\mathrm{B}_1\mathrm{B}_2\mathrm{T}_2\mathrm{T}_1$.

8. For practical reasons, the final search window is defined by a rotated rectangle rather than the rotated trapezoid $\mathrm{B}_1\mathrm{B}_2\mathrm{T}_2\mathrm{T}_1$. The rotated rectangle is a bounding box of the trapezoid. First, the distances $||\mathrm{B}_1\mathrm{B}_2||$ and $||\mathrm{F}_1\mathrm{F}_2||$ are compared. If $||\mathrm{B}_1\mathrm{B}_2|| > ||\mathrm{F}_1\mathrm{F}_2||$, the left and right sides of the rotated rectangle are passing through the points B_1 and B_2 respectively, otherwise they are defined by F_1 and F_2 . This ensures that the trapezoid search window cannot be clipped by the the rectangular search area.

3.3 Processing of search windows

3.3.1 Extending the search windows

In sections 3.2.1 and 3.2.2 we defined our punctuation search windows as rectangular regions that spread between in a text line area. However, this definition has not taken into account that many punctuation symbols, like commas or quotes, actually lie below or above this tight region. The simplest way to deal with this is to extend the search window height by a factor of constant before further processing.

It's possible to estimate the reasonable augmentation width and height using the following rule: Let's denote the width augmentation factor by w , and height augmentation factor by h . Let's assume that captured letters and their parts (including diacritics) are an important source of false positives. The number of captured letters depends both on w and h . A range of discrete values in range 1.3 of w and h has been searched exhaustively, the discretization step was 0.05, effectively resulting in $41^2 = 1681$ distinct vectors $[h, v]^T$. The number of captured punctuation marks TP as well as the number of captured letters FP has been evaluated for each of the parameter vector. Then we can define a cost function, which we want to minimize:

$$\underset{h,v}{\operatorname{argmin}}\ FP - \epsilon \cdot TP$$

The parameter ϵ is used to balance the desired trade- off between precision and recall. Based on experience with the rest of the punctuation pipeline, namely the success rate of the punctuation prefilter, the function was evaluated with $\epsilon = 0.1$, and the optimal augmentation factors on the dataset used were $[h, v]^T = [1.525, 1.625]^T$.

3.3.2 Extraction of search window images from a source image

The next step is to cut out the part of the original image specified by each of the search windows and compensate the slant. For each of the search windows, an up- right bounding rectangle is computed and the image region defined by this rectangle is cut out. The center of the up- right rectangle is identical to the centre of the augmented region and, therefore, we can rotate the image that was cut out about it's centre in angle opposite to the original search window rotation. After trivial clipping, we obtain an up- right (de- rotated) image region that was located within the augmented search window. This step improves the rotational invariance and significantly simplifies the further processing.

A special case when the augmented search window might extend beyond the region of the image is taken into account. In such case a part of the search window that's out of the image is filled with a constant black background. Since this area is always touching the borders of the augmented search window, it will be neglected during the connected component analysis step and can't introduce any artifacts or false positives.

3.4 Processing the information in a search window

The goal of the segmentation stage is to obtain a binary image of a search window. First, the color image is converted to single- channel grey- scale image. The words neighbouring the search window are known and threshold values for each of their respective letters are known, as well.

It is reasonable to expect that the ink color of a punctuation mark will not be very different from the ink of neighbouring words. It is possible to use a straightforward approach: If a search window has only one neighbouring word (the case of leading/trailing punctuation), choose a letter contained in the neighbouring word, which is closest to the search window and use the threshold of the letter to obtain the binarized image. If a search window has two neighbouring words, take the average threshold from letters in both of the words.

Indeed, a better approach for determining the threshold could be used. The rule mentioned above tends to fail on occasion of shadows or continuous changes of illumination. The thresholds could be interpolated through the means of linear regression or similar model, which, if based on more than just one or two letters, might be more robust. This strategy wasn not employed in this work, but will be considered in the future.

Figure 3.5: A visual representation of the described process: from a search window to extracted connected components.

3.4.1 Filtering the connected components

A large number of connected components is produced during the segmentation stage of the method. The ratio of garbage blobs to useful blobs (which are parts of a punctuation mark) has been experimentally shown to be more than 99% (see the Chapter 8). The number of extracted blobs has to be reduced in order to make the further process of component grouping possible. The pre- filter stage is an essential part of the design, and its performance has a direct effect on the performance of the whole pipeline.

Since the notion of punctuation regions and component groups does not exist at this point yet, it is necessary to process the individual connected components, and classify each of them as potentially being/not being a part of a future punctuation region.

3.4.2 Punctuation pre-filter heuristics

First, a simple heuristic is applied:

> **Rule 3.4.1.** The connected components that are smaller than 3×3 pixels are removed.

This is justified by the fact that these components are usually a result of image noise or compression artefacts. In fact, if they were valid punctuation components, they would not be processed correctly since they lack of useful information about their shape.

Other rule prevents the algorithm from processing the components that are likely to contain incomplete information:

> **Rule 3.4.2.** Connected components touching the border of a search window are discarded from further processing.

The stage of punctuation window augmentation makes it possible that a detected letter appears within a punctuation area. The partially clipped letters are already excluded by rule 3.4.2, however, narrow letters (e.g. 1, l) sometimes appear in the punctuation detection window unclipped. Thus, another heuristic rule is defined to help remove these stray letters:

Rule 3.4.3. If a segmented connected component has more than 30% overlap with a region known to be a letter, it is considered to be an artefact introduced by the letter and, therefore, will be discarded.

3.4.3 Punctuation pre-filter classifier

Remaining components are binary- classified using following scale- invariant features. Some of the features have been proposed in [1] for character/non- character detection and have been successfully used in this task, as well. Other features have been added to describe the position of a punctuation region component on a text line.

- Aspect ratio of the component bounding box $\frac{h_r}{w_r}$ [1],
- Compactness, $\frac{P^2}{s_r}$ [1],
- Ratio of convex hull to region area $\frac{C_r}{s_r}$ [1],
- Number of defects (convexity infex points) on the region contour, [1]
- Ratio of hole area and component area $\frac{H_r}{s_r}$
- Ratio of pixel- wise area of the component to area of the tight up- right bounding box of the component.
- Width of the component relative to height of the text line.
- Vertical distance of the lower- most point of the component from a bottom text line, relative to height of the line.
- Vertical distance of the upper- most point of the component from a bottom text line, relative to height of the line.

Before further processing, the PCA [12] dimensionality reduction technique takes place. It has been experimentally verified that this classifier performs better if dimensionality is reduced to a 7d space, since there is some correlation between the proposed features. The feature space is normalized before the PCA step: Each component x_i of a feature vector $x \in \mathbb{R}^4$ is computed as $norm(x_i) = (x_i - \bar{X}_i)/S_i$, where S is a vector of standard deviations and $\bar{X}$ is a vector of mean values for a given dataset X .

A ν - SVM classifier [13] with an RBF kernel is then used to perform classification in the resulting feature space. The σ and ν parameters were chosen from a discretized range of values, based on 8- fold cross- validation results. The cross validation error of the classifier is less then one percent.

3.4.4 Training set of the punctuation pre-filter

The training set consists of positive and negative examples. Positive examples have been synthesised from 58 common fonts. Negative examples have been extracted from various real- scene images. Not whole glyphs, but individual components are considered during the training, because mutual relations of components are not known yet. Since the used features are not fully scale invariant in practice (especially in low resolutions), all synthetic glyphs have been rendered in both high and low resolutions.

3.4.5 Grouping the connected components

The process of component grouping that will be introduced is simplistic and the success rate is dependent on the quality of the previous process, especially the prefilter. Looking at the Figure 2.2 we can easily tell that most of the multi- component punctuation consists of components that overlap vertically. The exception is the double prime (inches) and perhaps quotes (depending on a font). The following heuristic rule works for most of punctuation marks:

> **Rule 3.4.4.** Two components are grouped together iff their x - axis projection overlaps.

3.5 Punctuation classifier

After the groups of connected components have been established, it is possible to classify the resulting objects. Based on the practical experience about their frequency, the following classes have been chosen to be supported by the classifier prototype: dash, dot, comma, colon, semicolon, slash, question mark and exclamation mark.

Figure 3.6: The comma. It is interesting to notice that even in a symbol this simple, the placement and shape can be rather varied.

The classification is based both on shape and position of the regions. The following features are used in the classifier:

- A chaincode shape vector based on [1] and [2],
- Relative y- axis position of lowermost point of the region,
- Relative y- axis position of uppermost point of the region.

3.5.1 Computation of the shape vector

Figure 3.7: A blurred chain code representation of an exclamation mark glyph. Every 4×4 pixel patch of these masks creates one feature in the final vector.

The idea of using the chain codes to represent shape of a punctuation region, as well as the algorithm of normalization and chain code computation, is taken over from [1] and [2], where the same methods are used for extraction of letter features. The principles of this method will only be described briefly here.

Since the chain codes are to be computed from an image of a fixed size, 20×20 in this case, the components have to be scaled to match the size. The mask image is scaled so that after the scaling the larger dimension of width and height of the mask image is equal to 20 pixels and the other dimension is computed to preserve the aspect ratio. Then the scaled mask is copied into a matrix of size 20×20 so that it is centred.

A chain- code of the character perimeter is generated and, for each chain- code direction, the corresponding pixels are extracted into a separate bitmap. On each bitmap a regular 7×7 Gaussian masks are evenly placed to generate 25 features. In total, 25 features $\times$ 8 directions generate 200 features for each of the connected components. [2]

3.5.2 Dimensionality reduction

The feature vector used in this task consists of 202 features. Chain code vectors are sparse, and not all of their components have the same discriminative power (and the optimal weights for each of the features are not known). A combination of Principal Component Analysis (PCA) [12] and Linear Discriminant Analysis (LDA) [12] is used to solve this problem.

First, the dimensionality of the chain code shape vector is reduced with PCA. The number of principal components has been experimentally set to 70, so that the LDA problem is no longer singular or close to singular in the following step.

The additional features describing the position relative to the baseline are added to PCA coordinates, and the whole matrix is further processed with LDA in order to obtain a $k-1$ dimensional feature space (where k is the number of classes).

3.5.3 The classifier

Classification is performed using a straightforward 1- Nearest Neighbour rule. The 1- NN strategy can be successful in this particular case, because the data are artificial, noise-free and well- separated in the described feature space.

The classifier is trained on synthetic data described in Section 3.4.4 (positive examples only). However, the interpretation of the data is different here. While in Section 3.4.3 individual connected components of a glyph are extracted and trained separately, in this case, whole (potentially multi- component) glyphs are used for training.

This classifier is used to classify every detected punctuation mark. The problem of updating the final OCR classification with resulting punctuation components will not be discussed in this work, however, it should be very straightforward. Since the text line the punctuation mark belongs to is known as well as the neighbouring words, punctuation marks can be inserted easily.

Figure 3.8: A preview on the three most significant components of LDA-transformed feature space used in the classifier. Different classes are almost separated in this 2-dimensional projection. The real classification takes place in 7-dimensional LDA space.

Figure 3.9: A visualisation of punctuation pipeline result. Search windows are depicted in yellow, found regions and their classifications are in red.

3.6 The flow of the algorithm

Figure 3.10: An overview of the Punctuation pipeline.

Chapter 4

Diacriticals

This chapter focuses on detection and recognition of diacriticals in Czech language. Most of principles described here can be easily extended and adapted for different languages. The method described in this chapter uses the following information:

Input: a real- scene image a list of detected textlines, words and letters (their locations and segmentation) the output of letter OCR
Output: a list of detected diacritical pairs (letter, diacritical mark) and classification of diacritical marks

The algorithm flow can be described in few basic steps:

1. Analyze each output word from the TextSpotter and locate diacritical search regions.
2. Perform binary thresholding of diacritical search regions.
3. Filter connected components obtained in the previous stage and keep only these that pass the heuristic rules and are likely to be a diacritical mark to some of letters according to a classifier.
4. Classify the diacritical regions.
5. Adjust final OCR output according to classifications obtained in previous step (described in detail in Chapter 5).

4.1 Diacritical search windows

Searching the whole image would be too wasteful and the number of false positives would be very high. However, it is possible to use the information about letters and a text line to define a search region above each of the letters. Then, diacritical mark candidates (regions that could be diacritical marks) are is being sought.

Figure 4.1: Definition of a diacritical search region. H - text line height, w - distance from axis of the letter bounding box from a base line, b distance of the bottom border line of the search region from a base line.

The diacritical search region is an up- right rectangular area. Parameters of this search region are:

- w - horizontal span of the rectangle

- b - distance from a text bottom line to lower border of the search window
- t - distance from a text bottom line to upper border of the search window

All the measurements are relative to the line height H .

The actual parameters of the windows have been extracted from a training set consisting of rendered glyphs of letter with diacritical marks that are in use in Czech alphabet. They have been adjusted so that all diacritical marks in the dataset were captured in this region. The search windows were then augmented by 15% to compensate the lack of noise in synthetic data.

The obtained results are following:

parameter	value on synthetic data	adjusted value
t	1.5 H	1.725 H
b	0.675 H	0.776 H
w	0.486 H	0.559 H

Table 4.1: Diacritical search window parameters.

This heuristic ignores dimensions of the letter above which a diacritical mark is being sought. The text line height is considered to be a robust estimate of a font scale and all the measurements are relative to a text line height.

4.2 Segmentation of the search windows

Next, we have to perform segmentation in the search window. The rectangular region of an image is cut out, converted to a single- channel grey- scale and binarized. Since we assume that the color of a diacritical mark will always be close to a color of it's respective letter, the same value of threshold is used.

Processing of obtained binary image is similar to processing of punctuation search windows in section 3.4.2. The rules 3.4.1 (removal of too small components) and 3.4.2 (removal of components touching the border) are applied.

In Czech language, diacritical marks are always single- component glyphs. This assumption is wrong for glyphs like umlaut present in German though. If such multi-component diacritical marks were present in a target language, a simple strategy similar to rule 3.4.4 should be sufficient to group them together. This technique has not been tested in this work though.

The process described above is repeated for each of the letters of the input word separately. The result of this stage is a list which contains pairs. Each of the pairs consists of a letter region and a list of regions, that are diacritics candidates for this letter.

4.3 Detection of diacritical pairs

Although the search region above a particular letter is relatively small compared to area of the image, and the spatial constrains imposed by it's position and size are already sufficient to filter out most of the potential false- positives, it proved necessary to employ an additional detection stage to improve the precision of the algorithm.

"Being a diacritical" is a pair- wise relation between two regions: a letter and a diacritical mark. The information about a shape of a diacritical mark alone is insufficient for a precise diacritical/non- diacritical classifier, because diacritical glyphs are too simple and can be confused other objects around a text line. The search regions above letters are wide enough to overlap mutually and may contain diacriticals of neighbouring letters within the same word. So the diacritical detector presented in this chapter has to use pair-wise features between a region which is known to be a letter and a each of its diacritical candidates.

Figure 4.2: Anchor points in diacritical detection.

First, so called "anchor points" are determined. Diacritical anchor point First, denoted as $\mathbf{P}_D$, letter anchor point, denoted $\mathbf{P}_L$. Their definition is the following:

> **Rule 4.3.1.** To locate a diacritical [letter] anchor point, the bottom- most $\frac{2}{3}th\left[top - most\frac{1}{5}th\right]$ of scan lines in a diacritical [letter] glyph are determined. The anchor point of a diacritical [letter] lies in the center of mass of a region within these scan lines.

Then, the anchor points are then used as reference points to compute following geometric features:

- An angle of a line segment connecting $\mathbf{P}_T$ and $\mathbf{P}_D$, relative to $\mathbf{Y}$ axis,
- A magnitude of the vector $\mathbf{P}_T - \mathbf{P}_D$ relative to line height,
- Vertical distance of the letter and diacritical candidate (relative to line height),
- Ratio of areas of both regions.

The suggested features are scale invariant. They are also quite insensitive to rotation, although not completely rotationally invariant. This issue could be further addressed by estimating slant of a particular line and applying an inverse transform before computation of the feature vector takes place.

The feature space is then normalized. As in punctuation classifier, each component x_i of a feature vector $x \in \mathbb{R}^4$ is computed as $norm(x_i) = (x_i - \bar{X}_i)/S_i$, where S is a vector of standard deviations and $\bar{X}$ is a vector of mean values for a given dataset X . The training set consists of both positive and negative examples extracted from a training dataset consisting of annotated real- world images. It contains 270 true diacritical pairs and 133 negative examples.

A three- dimensional projection of the detector's feature space can be seen on Figure 4.4. This feature space is used to classify every connected component extracted from a diacritical window in the previous steps. Both linear ν - SVM, RBF kernel ν - SVM and trivial 1- NN rule classifiers have been tested, yielding comparable results. A linear ν - SVM was chosen for final experiments. Parameters ν and γ for SVM classifiers were estimated by exhaustive discrete search on a 2D grid of a 10- fold cross- validation.

This classifier is used to assign a respective diacritical mark from the list of candidate regions obtained in the previous step to each of the letters. Each letter is assigned at most one diacritical region.

If more than one candidate regions are segmented above the same letter, and more than one are classified as diacritics, the one having a better classification score feature

space is taken. Naturally, for a 1- NN classifier the score is basically the distance to the nearest neighbour. In a SVM classifier the score is a probability estimation score from libSVM (described in [14], Section 8, Probability Estimates).

4.4 Classification of diacritical regions

Figure 4.3: A graphical explanation of the features used in the diacritics classifier.

After the diacritical pairs are have been established, each of diacritical regions has to be classified so that we can use this information to improve the result of the OCR algorithm. There are four kinds of diacritical marks in our dataset (acutes, carons, rings and dots). Since hook carons are very rare, they've been neglected in this work. The diacritical marks are much simpler than punctuation marks, and a simpler feature space has been chosen. In this feature space, each diacritical glyph is described by four features:

- A ratio of major and minor axes of an ellipse that fits (in a least- squares sense) the set of diacritical region pixels best at all. This feature is chosen to capture the differences in aspect ratios of different diacritical marks
- An angle of main axis of the ellipse defined above, relative to X axis. This feature is chosen to capture the slanted nature of "acute" symbol.
- Reflection symmetry, the axis of symmetry is vertical. The glyph of the diacritical mark is flipped about the vertical axis and the ratio of intersection and union with the original glyph is computed. This feature well represents the symmetric/non-symmetric nature of different diacritical marks.
- Convexity defect ratio, i.e. the ratio between the area of the region and convex hull of its contour. High convexity defect ratio is observed in carons, other diacritical symbols are usually more or less convex.

These features are scale invariant. The lack of rotational invariance, again, could be compensated using the information TextSpotter is able to provide about rotation of a particular word if the performance of the diacritical processor was not satisfactory.

Figure 4.4: A 3-dimensional preview of diacriticritical detector feature space.

Figure 4.5: A visualisation of the diacritical detection and recognition output. The letters from the TextSpotter system are depicted in light orange, the detected diacritical regions in green. Diacritical search windows are marked with rounded overlapping cyan rectangles. Detected diacritical pairs are bounded by red rectangles.

4.5 The flow of the algorithm

Figure 4.6: An overview of the Diacritical pipeline.

Chapter 5

Integrating the information about diacritical marks with letter classification

In the previous chapter a method for detection and recognition of diacritical marks above letters detected by the OCR system has been introduced. In this chapter we will discuss how to use the information about diacritical marks to adjust the final OCR classification so that the information gathered about diacriticals is reflected. First, a trivial method will be introduced, then the possibility of implementation of a more elaborate HMM model will be discussed.

5.1 The implemented solution

Because the classification of every particular letter is known as well as the classification of the diacritical region that belongs to it, it's possible to use a simple table- of- replacements approach. Albeit not ideal, this approach has been implemented in this work because it is technically simple and does not require any additional training data.

The method uses the following information:

1. The letter OCR classification, which is assumed to be known and correct.
2. An information about a diacritical mark detected above the letter and if it was detected, its class.
3. A table of replacements similar to Figure **??**.

Having a letter and the result of diacritical detection and classification, the method proceeds by looking up the particular combination of a letter classification and OCR classification in the table of replacements and performing the replacement according to the table.

The table of replacements can be designed to reflect common cases of diacritics classifier confusions (i.e. according to confusion matrix in Chapter 7 a dot and acute confusion is quite common) as well as to take into consideration common OCR mistakes (i.e. "1" with a dot is likely to be an "i"). The results in Table 7.4.3 prove that even this trivial approach is indeed able to improve the OCR results.

5.2 A discussion of an alternative solution based on HMM

The above- mentioned table- of- replacements solution is computationally efficient and very easy to implement and understand, however, there are severe limitations:

1. It is a hand-tuned heuristic solution.
2. The information about the OCR classification is expected to be always correct, except for special explicit cases, eg. the $1^n + dot =$ "i" rule.
3. It is ignorant to prior and posterior probabilities of both letters and diacriticals as well as the surrounding letters.

All these problems could be effectively solved if a Hidden Markov Model was used. In this section a draft of this HMM will be described.

An HMM can be used to estimate the joint probability

$$P((l_1, d_1), (l_2, d_2), \ldots, (l_n, d_n), y_1, y_2, \ldots, y_n)$$

where $l_x \in A = \{a, b, \ldots, z, A, B, \ldots, Z, 0, \ldots, 9\}$ (i.e. Roman alphabet without diacriticals), $dx \in D = \{nothing, acute, caron, ring, dot\}$ (i.e. diacritical marks) and $y_x \in \bar{A}\{a, \dot{a}, \ldots, \bar{z}, A, \dot{A}, \ldots, \dot{Z}, 0, \ldots, 9\}$ (i.e. alphabet with diacritical marks). The values of l_x and d_x are the outputs of the original OCR and the diacritical pipeline respectively. Therefore, the task would be to find

$$\underset{y_1 \ldots y_n}{\operatorname{argmax}} P((l_1, d_1), (l_2, d_2), \ldots, (l_n, d_n), y_1, y_2, \ldots, y_n)$$

for fixed tuples of (l_x, d_x) . In case of an HMM, this can be efficiently done using the Viterbi algorithm [15].

The parameters of a HMM consist of two matrices:

- ϕ_{y_i, y_j} matrix of probabilities of transitions from a state (letter) y_i to y_j
- $\theta_{y_i, (l_j, d_k)}$ matrix of probabilities of emissions of tuples (l_j, d_k) in state y_i

The matrix ϕ can be easily estimated from a language corpus, for example [16]. However, the matrix θ can not be directly obtained, since the matrix θ is, in fact, a matrix of conditional probabilities $P(l_i, d_j | y_k)$. The number of items in this matrix would be $A \times D \times \bar{A}$; it is obvious that the amount of data needed to estimate the probabilities directly would be immense, requiring extremely large annotated training sets.

However, this problem can be made tractable under the assumption that the posterior performance of letter classifier and diacritical detection and classification cascade are independent, i.e. we assume it holds that:

$$\forall l_i \in A, \forall d_j \in D, \forall y_k \in \bar{A} : P(l_i, d_j | y_k) = P(l_i | y_k) \cdot P(d_j | y_k)$$

If independence holds, we do not need to estimate joint probabilities $P(l_i, d_j | y_k)$ directly, instead it is sufficient to measure the expected performance of letter classification, diacritical detector and diacritical classifier, namely confusion matrices, per- class accuracies and false- positive rates.

Let's call the letter OCR confusion matrix $\mathbf{C_L}$ and diacritical classifier confusion matrix $\mathbf{C_D}$. The confusion matrix $\mathbf{C_D}$ implicitly contains the information about the number of false positives and false negatives (since we added the "nothing" class). We assume that each row of such a matrix contains confusion frequencies for a given class, i.e. every row sums to 1. Under the assumption that the classifiers are independent, the matrix θ can be constructed as follows:

$$\theta_{i,(j-1)\cdot n+k} = \mathbf{C_L}_{f(i),j} \cdot \mathbf{C_D}_{g(i),k}$$

where

- $i \in I_{\bar{A}}$, an index of a letter in $\bar{A}$ (alphabet with diacriticals)
- $j \in I_A$ an index of a letter in A (alphabet without diacriticals)
- $n = |D|$ (number of diacritical marks including "nothing")
- $k \in I_D$ an index of a diacritical mark in D
- f a mapping $I_{\bar{A}} \to I_A$, ie. the mapping assigning a letter without diacriticals to a letter with diacriticals
- g a mapping $I_{\bar{A}} \to I_D$, ie. the mapping assigning a diacritical mark to a letter with diacriticals

Having this HMM, the output of the letter OCR and diacritical pipeline would then be processed by translating the classifications to respective indices from I_A and I_D and processing the resulting chain using the Viterbi algorithm to obtain the most likely explanation for observed values.

Figure 5.1: An illustration of the suggested HMM for integration of letter and diacritical classification. Only selected states, outputs and edges are portrayed. Inspired by [17].

The suggested method has not be tested, because the confusion matrix of the TextSpotter OCR was not available at the time of writing this thesis. The TextSpotter evaluation protocol (as well as the official ICDAR evaluation protocol [18]) focuses on words rather than letters and cannot provide reliable information needed to build a confusion matrix. This technical issue will be addressed in a future work.

Chapter 6

Datasets and testing

In this chapter the datasets used to verify functionality of the proposed methods are described more in detail.

Although more standard datasets designed for testing of text- in- the- wild recognition algorithms are available, for example a dataset published for ICDAR 2003 Robust Reading Competition [18] or Chars74K dataset [19], they didn't offer enough opportunities to test recognition of diacritical and punctuation marks. Also, these datasets don't contain annotations detailed enough for our purpose. Different datasets, specifically focusing on this problem were needed.

6.1 Characteristic of the used datasets

Two new real- scene datasets, specifically tailored for testing and verification of methods for both punctuation and diacritical marks recognition are proposed in this work. The datasets contain scenes from streets, featuring various labels, advertisements and posters.

The images are deliberately photographed from varying angles, under non- ideal light conditions, many include combinations of more font styles, ink colors and text line scales in a single image. They were all taken using a cheap low- end camera or a mobile phone, in order to simulate the expected condition of input images in a real application. The texts on the image are mostly in Czech language.

name	images	words	letters	punc. marks	dia. marks
CS Dataset2	40	709	3305	84	452
CS Dataset4	40	459	3029	249	349

Table 6.1: Statistics of the datasets.

All the presented datasets contain detailed hand- made annotation, which consists of up- right tight bounding boxes around all the regions of interest (ie. letters, diacritical marks and punctuation marks) labelling of these fields and grouping of them into words.

CS Dataset2, and CS Dataset4 are both available for public use.

Figure 6.1: An example showing the detailed annotation. The word rectangles are depicted in white, letter rectangles in green and diacritical rectangles in yellow. Letter and diacritical rectangles are only shown in the first word.

Figure 6.2: Example from the datasets. This image is featuring punctuation and challenging reflections.

Figure 6.3: Example from the datasets. Note the punctuation, diacritical marks, different scales and lens distortion.

Chapter 7

Evaluation of a diacritical pipeline

In this chapter an algorithm used to assess the performance of a diacritical pipeline will be introduced. The algorithm is based on comparing annotated bounding rectangles of letters and diacritical regions with results obtained from an algorithm cascade described in the previous chapter.

Figure 7.1: The flow of the evaluation algorithm.

7.1 Description of individual evaluation stages

The overall view on the evaluation routine is presented on the Figure 7.1. In this section, individual stages of the routine will be described more in detail.

The algorithm starts by running the TextSpotter and the diacritical pipeline described in Chapter 4 on an image. After the computation, the following pieces of information are gathered:

- bounding rectangles of letters detected by the TextSpotter (tsLetters)
- OCR representation of a word (plaintext)
- a list of connected components detected in Section 4.2 (segmentedRegions)
- a list of diacritical relations detected in Section 4.3 (detectedDiaRelations)
- a list of classifications obtained in Section 4.4 (also detectedDiaRelations)

The data mentioned above are compared with referential annotation data consisting of list of hand- annotated or semi- automatically annotated letters (bounding rectangles and classification of both letters and diacritical marks).

7.1.1 Procedure identifyLetter

Tested component: the TextSpotter system itself

Input: annotatedLetter tsLetters

Output: tsLetter, if a good match found, nothing otherwise

This procedure is performed for every letter in the list of annotated letters. The goal is to find a corresponding TextSpotter letter, ie. a letter that has the best bounding box

match. Let A is an annotated letter bounding box and B is a bounding box of a letter detected by the TextSpotter. The bounding box match score is computed as follows:

$$\textbf{match} = \frac{area(A \cap B)}{area(A \cup B)}$$

Given an annotated letter, this score is computed for every letter in the TextSpotter output. The letter with highest matching score is chosen. The score of this letter is then compared to a cut- off value, which was manually adjusted to capture the intuitive notion of a "good match". Which means, that the result of automatic evaluation is very close to results achieved by evaluating the results fully manually. The value found to work the best for this purpose was 0.6. If the match value for the best- matching letter is greater than 0.6, the letter is considered to be found and is kept for further processing.

7.1.2 Procedure identifyDia

Tested component: segmentation algorithm (described in 4.2)

Input: annotatedLetter segmentedRegions

Output: dia if a good matching connected component found, nothing otherwise

This procedure tries to find the best matching segmented region from the segmentedRegions, given a diacritical mark of an annotatedLetter. The algorithm is analogous to the previously mentioned one. The difference is that the score threshold value is set to only 0.25, since diacritical marks in the datasets may be as small as 4×4 pixels and precise annotation is very difficult due to blur. Although this value seems to be very small, the practice shows that the number of falsely accepted components is still negligible, but the number of wrongly rejected correct results is minimized at this threshold.

7.1.3 Procedure identifyDiaRelation

Tested component: diacritics detector (described in 4.3)

Input: tsletter dia detectedDiaRelations

Output: diaRelation or nothing if the relation not detected

The goal of this procedure is to determine which of detectedDiaRelations corresponds to a given pair of tsLetter and dia obtained in previous two steps. The structure detectedDiaRelations is a map of found letters $\rightarrow$ found diacritical marks. The tsLetter is used as a key. If the diacritical mark obtained in this step matches dia, the corresponding diacritical relation is returned.

7.1.4 Check the classification and the result of rewriting

Since we have already found all annotated/detected pairs we need, the last step is to the classification results with the annotation and to assess the final outcome (letter rewritten using the information available from the diacritical pipeline). The diaRelation obtained in the previous step contains both pieces of information we need. The labels of a diacritical mark and the resulting ASCII codes are compared and the statistics are update accordingly.

7.2 Statistics gathered during the evaluation process

- Number of letters detected by the TextSpotter
- Global performance
  - Total number of segmented connected components (CCs) (all).
  - Total number of CCs correctly detected as diacritics (TP).
  - Total number of CCs correctly rejected from processing (TN).
  - Total number of CCs components incorrectly detected as diacritics (FP).
  - Recall $\left(\frac{\mathrm{TP}}{\mathrm{TP+FN}}\right)$
  - Precision $\left(\frac{\mathrm{TP}}{\mathrm{TP+FP}}\right)$
  - Accuracy $\left(\frac{\mathrm{TP}}{\mathrm{TP+FP}}\right)$
  - F_1 -measure
- Per-class true-positive statistics at each stage of the pipeline.
- Confusion matrix of the classifier.
- Re-writing performance.

7.3 Statistics of a dataset

Number of images. Total number of words. Total number of letters. Total number of diacritical marks (acutes, carons, rings and dots).

7.4 Results

7.4.1 Dataset

Number of images	40
Number of words	709
Number of letters	3305
Number of diacritical marks	452

Table 7.1: Dataset statistics for diacritical evaluation.

7.4.2 Performance on connected components

7.4.3 Per-class detailed statistics

7.4.4 Classifier confusion matrix

all	677	
TP	171	(25.3%)
TN	449	(66.3%)
FP	20	(3.0%)
FN	37	(5.5%)
Recall	0.822	
Precision	0.895	
F-measure	0.857	

Table 7.2: Performance on connected components.

	all	acute	caron	ring	dot
letter found	331	154	123	11	41
segmentation	208 (62.8%)	96 (62.3%)	76 (61.8%)	7 (63.6%)	29 (70.7%)
detection	171 (82.2%)	79 (82.3%)	65 (85.5%)	4 (57.1%)	23 (79.3%)
classification	156 (91.2%)	74 (93.7%)	65 (100.0%)	2 (50.0%)	15 (65.2%)
rewriting	160 (102.6%)	76 (102.7%)	62 (95.4%)	4 (200.0%)	18 (120.0%)

Table 7.3: Per-class detailed statistics.

reality/classifier	acute	caron	ring	dot
acute	0.433	0.00585	0	0.0234
caron	0	0.380	0	0
ring	0	0	0.0117	0.0117
dot	0.00585	0	0.0409	0.0877

Table 7.4: Confusion matrix of the diacritical classifier.

Chapter 8

Evaluation of a punctuation pipeline

This chapter will describe the method used to evaluate the performance of punctuation detection and recognition. The evaluation method is very similar to method described in Chapter 7.

Figure 8.1: The flow of the evaluation algorithm.

8.1 Description of individual evaluation stages

The overall view on the evaluation routine is presented on the Figure 8.1. As with diacriticals, the bounding boxes of detected regions are compared with hand- made annotations. However, since punctuation marks can consist of multiple connected components, care must be taken while handling these. The whole punctuation pipeline has been tested on five most common classes of punctuation marks - dashes, dots, commas, colons and slashes.

The algorithm uses the following input data:

- a list of punctuation search windows (searchWindows [])
- a list of output punctuation regions (foundPunc [])
- a list of connected components obtained during the segmentation stage, ie. after steps described in Section 3.3.2 (foundCCs [])
- a list of connected components left after prefiltering and grouping stages, ie. after Section 3.4.5 (acceptedCCs [])
- a number of recognition misses/hits, a confusion matrix

8.1.1 Procedure inWindows

Tested component: the TextSpotter system and the rules defining search windows

Input: annotatedPunc - an annotated punctuation region, windows [] - a list of punctuation search windows

Output: true if the annotatedPunc is within a window, false otherwise

This procedure checks for a given annotated punctuation region whether it has been captured in one of punctuation search windows or not. The check is very simple: for each of punctuation windows check whether all four corners of the annotated punctuation region lie within the area of the search window. If at least one window is found to satisfy the condition, the annotated region is considered to be reachable by our method and this procedure returns true.

8.1.2 Procedure findBestCCs

Tested component: segmentation or prefilter, depending on input

Input: annotatedPunc - bounding box of an annotated punctuation mark, ccs [] - a list of connected components to look for the matching with

Output: a list of connected components that have the best coverage with the annotatedPunc or nothing if there is no coverage or the coverage is below the threshold

The goal of this evaluation procedure is to choose a subset from a list of all given connected components (defined as pairs of bounding rectangles and glyphs) that have the best overlap with a given annotatedPunc. The method following strategy is used:

1. An intersection of all of ccs with the annotatedPunc is computed.
2. Since the ground truth classification of annotatedPunc is know, the number of components it should be covered with (let's call it n) is known too.
3. Choose n components with best intersection with the annotatedPunc, if there are less than n components with non-zero intersection, the procedure fails at this point.
4. Take a set of corner points of components chosen in the previous step. Construct a convex hull of the points - a polygon H .
5. Compute the ratio
$$\text{match} = \frac{area(H \cap P)}{area(H \cup P)}$$
where P is the bounding box of annotatedPunc.
6. If match > 0.6 , return the list of n components with best coverage. Otherwise return nothing.

8.1.3 Procedure checkGrouping

Tested component: the grouping heuristic

Input: ccs [] - a list of connected components that should be grouped according to annotations and the previous procedure, punctuation - a list of detected punctuation regions

Output: a detected punctuation region which corresponds to given ccs

This procedure is used to evaluate the performance of the grouping heuristic. It accepts a list of connected components assigned to an annotated punctuation region in the previous stages and verifies whether the components have actually been found as a part of a detected punctuation region. If the procedure succeeds, the corresponding punctuation region is returned. Otherwise it returns false.

8.2 Statistics gathered during the evaluation process

- Number of words detected by the TextSpotter
- Number of punctuation search windows
- Global performance
  - Total number of segmented connected components (CCs) (all).
  - Total number of CCs correctly detected as a part of a punctuation symbol (TP).
  - Total number of CCs correctly rejected from further processing (TN).
  - Total number of CCs components incorrectly detected as a part of a punctuation symbol (FP).
  - Total number of CCs components incorrectly rejected from further processing (FN).
  - Recall $\left(\frac{\mathrm{TP}}{\mathrm{TP+FN}}\right)$
  - Precision $\left(\frac{\mathrm{TP}}{\mathrm{TP+FP}}\right)$
  - F_1 -measure

8.3 Statistics of a dataset

Number of images. Total number of words. Total number of letters. Total number of punctuation marks (dashes, dots, commas, colons and slashes).

8.4 Results

8.4.1 Dataset

Number of images	40
Number of words	459
Number of letters	3029
Number of punctuation marks	233

Table 8.1: Dataset statistics for punctuation evaluation.

8.4.2 Detection performance on connected components

8.4.3 Number of true positive regions at different stages

all	12387	
TP	125	(1.0%)
TN	12170	(98.2%)
FP	34	(0.3%)
FN	58	(0.5%)
Recall	0.683	
Precision	0.786	
F-measure	0.731	

Table 8.2: Performance on connected components.

stage \ class	all	dash	dot	comma	colon	slash
total	233	71	80	40	34	8
in windows	167	54	58	33	16	6
segmented	143 (85.6%)	47 (87.0%)	48 (82.8%)	27 (81.8%)	15 (93.8%)	6 (100.0%)
prefilter ok	118 (82.5%)	41 (87.2%)	45 (93.8%)	21 (77.8%)	7 (46.7%)	4 (66.7%)
grouping ok	118 (100.0%)	41 (100.0%)	45 (100.0%)	21 (100.0%)	7 (100.0%)	4 (100.0%)
recognized	117 (99.2%)	40 (97.6%)	45 (100.0%)	21 (100.0%)	7 (100.0%)	4 (100.0%)

Table 8.3: Number of true positive regions at different stages.

Chapter 9

Implementation

In this chapter the implementation of the algorithms described in this thesis will be described more in detail. This chapter refers to files and folders located on the enclosed CD. A complete description of the CD content can be found in Appendix C.

The implementation part of this work consists of two independent modules that are tightly integrated with the TextSpotter system. The TextSpotter itself is written in C++ and relies heavily on the OpenCV [20] library for image processing and pattern recognition algorithms.

Diacritical and punctuation modules described in this work have been written in GNU Octave [21]. This system was chosen because it - unlike the commercial package Matlab - offers seem- less and automatic integration of C++ code in the scripting environment (through SWIG [22]). Also, the scripting nature of the language allows for rapid prototyping, and provides enough flexibility for experiments.

The libraries used besides TextSpotter and OpenCV:

- LibSVM - a library offering various modifications of the Support Vector Machine classifier [14]
- Statistical Pattern Recognition Toolbox for Matlab - an implementation of standard classifiers, machine learning utilities and plotting routines for Matlab [23]
- JsonCPP - a library for reading/writing data JSON data files [24]

9.1 Diacritical module

The diacritical module is divided into three basic sub- modules. Class DiacriticsProcessor coordinates the whole process of diacritical marks detection and recognition. It contains all the definitions of diacritical search windows and a visualisation utility. Class DiacriticsDetector encapsulates all rules and the classifier needed to detect diacritical marks. Class DiacriticsClassifier encapsulates the diacritical classifier.

Both DiacriticsDetector and DiacriticsClassifier have a method called train which is used to train the internal classifiers using a dataset consisting of black and white PNG images. The training phase is considerably fast, so no need to store the resulting models in files arose. It is possible to re- train every time the algorithm is ran, which is a practical feature for fast prototyping.

The diacritical module resides in folder src/mfiles/Diacrirics on the enclosed CD.

9.2 Punctuation module

The structure of the punctuation module is very similar to structure of the diacritical module. There are three basic submodules containing the program logic. The class PunctuationProcessor, which defines the punctuation search window, basic component filtering and component grouping rules. The class PunctuationPrefilter defines training and prediction routines of the punctuation prefilter classifier. The class PunctuationClassifier contains the punctuation mark classifier as well as the routines for dimensionality reduction.

As with the diacritical module, the punctuation module can be easily re- trained any time using a folder of negative and positive examples/images. The sources of the punctuation module are located in folder src/mfiles/Punctuation.

9.3 Training datasets of classifiers

There are four directories that contain training data for classifiers. For simplicity, they have been contained in the same folder along with Octave source codes.

- Diacritics/detector-training - This is a dataset for diacritical detector training. It consists of both positive and negative examples (stored in pos and neg subfolders respectively). Every grayscale image in the dataset consists of one letter component and one or more diacritical/nondiacritical components. The components are marked with different pixel values - a diacritical component has a value of 254, a letter component has a value of 254.

- Diacritics/classifier-training - The dataset used for diacritical classifier training. It consists of synthetic images of diacritical glyphs.

- Punctuation/prefilter-training - This is a prefilter training dataset. It includes subfolders pos and neg for positive and negative examples respectively. Every positive example image contains a capital letter Z along with a punctuation mark. The height of capital Z is used to recover the height of a text- line and the relative position of punctuation glyph. All negative examples consist of one or more connected components that are not punctuation marks. The height of the image canvas is important here. It is used to recover the height of the punctuation search window.

- Punctuation/classifier-training - This dataset is used for punctuation classifier training. It has the same format as the positive examples in the previous dataset - leading capital Z is used for scale and position estimation.

Chapter 10

Conclusion

In this work, an approach towards detection and recognition of two different kinds of auxiliary textual symbols in real- world images has been proposed, and an attempt was made to integrate the information about diacritical marks in the final OCR classification. The author of this work is not aware of any other work focusing specifically on the problem of detection of diacritical and punctuation marks in a photo- OCR system. Therefore, the results achieved in this work are setting a baseline for future methods focusing on this kind of problem.

This work has presented a system for detection and recognition of diacritical marks with promising results (the detector F- measure rate was more than 0.85). The method uses simple classifiers and small training sets, which makes it computationally inexpensive and easy to implement. The weakest point of the diacritical detection and recognition method has shown to be the segmentation part, which, however, was not a subject of this work, and a lot of potential improvement is expected here.

We have shown that the information about diacritical marks can be exploited to improve the accuracy of the letter OCR and a draft of a language model which would be suitable for this purpose was introduced. Since the dot symbol is can also be successfully detected and recognized by the proposed system, it is possible that the information about diacritical marks could be used to improve the OCR results on standard datasets (e.g. ICDAR) that only contain "i" and "j" letters.

The chapter about punctuation detection and recognition introduces a robust definition of punctuation search windows, areas where punctuation marks are expected to be found. A method of detection and classification of punctuation marks is proposed.

With F- measure of 0.731, the performance in punctuation mark was significantly worse than the performance of diacritical detection. Unlike the diacritical detection and recognition algorithm, the weakest point of the punctuation pipeline is the pre- filtering stage. Although the cross- validation accuracy of the pre- filter is very high, around 99.6% , the real detection results are not so good.

This can be explained by the fact that the real input data, unlike the training dataset (which consists of comparable number of positive and negative examples) is highly unbalanced in favour of negative class (compare TP and TN in the table above). The problem of detection of punctuation marks is apparently more difficult than detection of diacritical marks:

- Punctuation search windows are much larger than diacritical search regions and, therefore, more false positives are captured during the segmentation stage. This fact is obvious when the result tables and are considered.

- Shape and dimensions of punctuation marks are more varied (especially between classes) than shape and dimensions of diacritical symbols, which leads to more complex distribution of resulting feature space and complicated classification.

The recognition of punctuation marks, performed using the 1- NN rule in the LDA-reduced chain- code feature space, led to be very good results, achieving the accuracy of 99.1% (only one mistake on the whole dataset).

Another contribution of this work is creation of two new datasets of real- scene images containing auxiliary textual regions of both kinds, along with precise letter- wise annotations, including annotations of diacritical marks. The annotations have been created using a semi- automatic annotation editor, designed to be aware of diacritical marks, which was also created as a part of the assignment.

Besides all above- mentioned contributions, this thesis also presents few open questions, some of which will certainly be topics of future research:

- Compensation of rotation in the punctuation pipeline did not perform well, sometimes actually worsened the results. It seems that most of line tilt observed in real-scene images is not actually caused by camera rotation but by more complex perspective deformation. Methods to estimate text- line homography could improve overall performance of letter and punctuation recognition.

- A better approach towards disconnectedness in punctuation marks should be found, since a simple heuristic rule presented in Section 3.4.5 did not perform well. A sliding- window approach applied in punctuation search windows could address the issue of punctuation pre- filtering and grouping more effectively.

- A better segmentation strategy should be considered, since the bottle- neck of diacritical detection and recognition was the segmentation stage.

- More experiments with letter classification in LDA space should be performed, to verify the hypothesis, that since LDA performed so well in punctuation marks, it could improve both speed and accuracy of the TextSpotter letter OCR.

Appendix A

Dataset format

The format of ground truth files enclosed with the datasets is described in more details here.

A separate annotation file is provided for each of the test images. The name of this file conforms with xxx_gt.json, where xxx is a place-holder for the file name of the image. Alternatively, it's possible to combine more image-wise annotation maps in a single file, since the name of the image is stored. The internal ground truth file format is based on JSON [25] exchange format.

Following semantic units are recognized: letters, words and whole images. Example of an annotation file:


```
{
  "filename" : "P3102094.JPG",
  "words" : [
    {
      "text" : "Páníkové",
      "nodiaText" : "Panickove",
      "bbox" : {
        "height" : 32,
        "width" : 227,
        "x" : 653,
        "y" : 277
      },
      "diaBbox" : {
        "height" : 41,
        "width" : 227,
        "x" : 653,
        "y" : 268
      },
      "letters" : [
        {
          "bbox" : {
            "height" : 31,
            "width" : 22,
            "x" : 653,
            "y" : 278
          },
```

```
          "char" : "P",
          "hasDiacritics" : false
        },
        {
          "bbox" : {
            "height" : 24,
            "width" : 21,
            "x" : 679,
            "y" : 283
          },
          "char" : "á",
          "dia_type" : "acute",
          "diacritics" : {
            "height" : 9,
            "width" : 11,
            "x" : 685,
            "y" : 272
          },
          "hasDiacritics" : true
        }
        /* ... 7 more letters ... */
      ]
    }
    /* ... more words ... */
  ]
}
```

Some of the JSON fields can be considered redundant (i.e. not "normalized", in database terminology), since they can be derived from another fields (eg. the field nodiaText is functionally dependent on text, one-way mapping exists between these fields). This is meant to be a feature rather than a bug, since it allows to keep the evaluation scripts more simple. The dependencies between these fields are summarized in the following table:

field	depends on
word.bbox	word.letters[].bbox
word.diaBox	word.letters[].bbox, word.letters[].diacritics
word.nodiaText	word.text
word.text	word.letters[].char
word.letters[].dia_type	word.letters[].char
word.letters[].hasDiacritics	word.letters[].diacritics !== undefined

Table A.1: Dependencies between JSON fields.

The provided Annotator application is aware of these dependencies and will ignore the dependent fields when loading the file and re- create them while saving the file, effectively avoiding the possibility of breaking the internal consistency.

The annotation format doesn't have a special support for punctuation marks. These are treated as normal letters and can be marked as separate words or (when semantically

more appropriate) as a part of a word. The routines for automatic evaluation of punctuation recognition and detection described in this work don't distinguish between these options.

Appendix B

Annotation editor

For creating and modifying the annotated datasets some annotation utility was needed, since there was no existing tool that would fit the needs (especially which would a separate annotation of diacritical regions). The annotation editor presented in this appendix has been implemented in C++ , Qt framework [26] and OpenCV library [20], the JSON exchange format [25] is being handled by JsonCpp library [24].

The annotation editor features a minimal but functional graphical user interface which makes the process annotating as straightforward as possible.

B.1 Semi-automatic annotation

The annotation supports a semi- automatic mode which utilizes independent methods to help user position the annotation rectangles more precisely and quickly. The principle is following: User provides a loose rectangle around a word the text as well as the word text itself. The chosen rectangular area is cut- out and transformed to grey- scale. Then the software attempts to find a good segmentation threshold using Otsu method [27].

The image is thresholded and connected components extracted and passed through simplistic heuristic filter similar to heuristics in 3.4.2. Since the text of annotated word is provided in advance, the software knows how many connected components are to be found (including the diacritical marks). Using simple rules based on left- to- right ordering and assumptions about mutual sizes and positions of diacritical marks and their respective letters, it's possible to match the text provided by user to the connected components that were found quite reliably.

The result is presented to user, who can accept it, try to adjust the threshold manually or fall- back to fully manual annotation process.

Figure B.1: Visualisation of semi-automatic annotation process.

Appendix C

Content of the enclosed CD

- datasets/ - Datasets used for evaluation.
- hadacja2-bc.pdf - This text.
- src/3rdparty - External libraries.
- src/cxx - C++ helpers and base classes.
- src/cxx/Annotator - Source code of the annotator.
- src/generators - Python scripts used to generate synthetic datasets along with used fonts.
- src/mfiles - Octave source codes.
- src/swig - SWIG metaprogram and conversion routines.
- tex - LaTeXsources and images used to build this thesis.

Appendix D

A visual preview of the datasets

The next two pages contain a preview on all 80 images in both datasets used for experiments in this work.

D.1 CSDataset2

Figure D.1: Preview of CSDataset2 (images not shown).

D.2 CSDataset4

Figure D.2: Preview of CSDataset4 (images not shown).

Appendix E

A visual preview of detection results

The next two pages contain detection result of diacritical marks and punctuation marks respectively. Successfully detected objects are marked with green rectangles, missed objects by red rectangles.

E.1 Diacritical pipeline

Figure E.1: Example of diacritical detection results.

E.2 Punctuation pipeline

Figure E.2: Example of punctuation detection results.

Bibliography

[1] Lukas Neumann. "Vyhledavani a rozpoznavani textu v obrazech realnych scen". Czech Technical University in Prague, Faculty of Electrical Engineering, 2010. URL: http://cmp.felk.cvut.cz/~neumalu1/Neumann-thesis-2010.pdf.

[2] Lukas Neumann and Jiri Matas. "A method for text localization and recognition in real- world images". In: ACCV 2010: Proceedings of the 10th Asian Conference on Computer Vision. (Queenstown, New Zealand). Ed. by Ron Kimmel, Reinhard Klette, and Akihiro Sugimoto. Vol. IV. LNCS 6495. Heidelberg, Germany: Springer, Nov. 8-12, 2010, pp. 2067-2078.

[3] L. Neumann and J. Matas. "Real- time scene text localization and recognition". In: Computer Vision and Pattern Recognition (CVPR), 2012 IEEE Conference on. 2012, pp. 3538-3545. DOI: 10.1109/CVPR.2012.6248097.

[4] B. Epshtein, E. Ofek, and Y. Wexler. "Detecting text in natural scenes with stroke width transform". In: Computer Vision and Pattern Recognition (CVPR), 2010 IEEE Conference on. 2010, pp. 2963-2970. DOI: 10.1109/CVPR.2010.5540041.

[5] Xiangrong Chen and A.L. Yuille. "Detecting and reading text in natural scenes". In: Computer Vision and Pattern Recognition, 2004. CVPR 2004. Proceedings of the 2004 IEEE Computer Society Conference on. Vol. 2. 2004, DOI: 10.1109/CVPR.2004.1315187.

[6] Gabriel Ehrnst Grundin [Public domain], via Wikimedia Commons. Quadrilingual danger sign - Singapore (gabbe). URL: http://en.wikipedia.org/wiki/File:Quadrilingual_danger_sign_-_Singapore_(gabbe).jpg (visited on 04/19/2013).

[7] Wikipedia, The Free Encyclopedia. Punctuation. URL: http://en.wikipedia.org/wiki/Punctuation (visited on 03/15/2013).

[8] Typography Deconstructed. A comprehensive guide to the anatomy of type. URL: http://www.typographydeconstructed.com/ (visited on 04/19/2013).

[9] Wikipedia. Tittle Wikipedia, The Free Encyclopedia. [Online; accessed 4- December 2013]. 2013. URL: http://en.wikipedia.org/w/index.php?title=Tittle&oldid=571443086.

[10] Wikipedia. Dotted and dotless I Wikipedia, The Free Encyclopedia. [Online; accessed 4- December- 2013]. 2013. URL: http://en.wikipedia.org/w/index.php?title=Dotted_and_dotless_I&oldid=582219443.

[11] Max Naylor [Public domain], via Wikimedia Commons. Typography Line Terms. URL: http://commons.wikimedia.org/wiki/File:Typography_Line_Terms.svg (visited on 03/15/2013).

[12] Richard O. Duda, Peter E. Hart, and David G. Stork. Pattern Classification. 2nd ed. New York: Wiley, 2001.

[13] Pai- hsuen Chen, Chih- jen Lin, and Bernhard Scholkopf. A tutorial on ν - Support Vector Machines. 2005.

[14] Chih- Chung Chang and Chih- Jen Lin. "LIBSVM: A library for support vector machines". In: ACM Transactions on Intelligent Systems and Technology 2 (3 2011). Software available at http://www.csie.ntu.edu.tw/~cjlin/libsvm. URL: http://www.csie.ntu.edu.tw/~cjlin/papers/libsvm.pdf.

[15] Michael Collins. Tagging Problems and Hidden Markov Models. Course notes for NLP by Michael Collins, Columbia University. 2013. URL: http://www.cs.columbia.edu/~mcollins/hmms-spring2013.pdf.

[16] Praha Ústav eského národního korpusu FF UK. eský národní korpus - SYN. 9.4.2013. URL: http://www.korpus.cz.

[17] Tdunning [CC- BY- 3.0], via Wikimedia Commons. Hidden Markov Model. URL: https://commons.wikimedia.org/wiki/File:HiddenMarkovModel.svg (visited on 12/26/2013).

[18] Lucas Panaretos Sosa et al. "ICDAR 2003 Robust Reading Competitions". In: In Proceedings of the Seventh International Conference on Document Analysis and Recognition. IEEE Press, 2003, pp. 682-687.

[19] T. E. de Campos, B. R. Babu, and M. Varma. "Character recognition in natural images". In: Proceedings of the International Conference on Computer Vision Theory and Applications, Lisbon, Portugal. Feb. 2009.

[20] G. Bradski. "The OpenCV Library". In: Dr. Dobb's Journal of Software Tools (2000).

[21] Octave community. GNU/Octave. 2012. URL: http://www.gnu.org/software/octave/.

[22] SWIG. A code generator for connecting C/C++ with other programming languages. 2013. URL: http://www.swig.org/.

[23] Vojtch Franc and Václav Hlavá. Statistical Pattern Recognition Toolbox for Matlab. 2008. URL: http://cmp.felk.cvut.cz/cmp/software/stprtool/.

[24] Baptiste Lepilleur. JsonCpp library. 2013. URL: http://jsoncpp.sourceforge.net/.

[25] JavaScript Object Notation. 2013. URL: http://www.json.org/.

[26] Qt project. Qt Project. 2013. URL: http://qt-project.org/.

[27] "A Threshold Selection Method from Gray- Level Histograms". In: Systems, Man and Cybernetics, IEEE Transactions on 9.1 (1979), pp. 62-66. ISSN: 0018-9472. DOI: 10.1109/TSMC.1979.4310076.